

Quoridor – Python

Rapport préliminaire

AIT-ISSAD MOHAMED SAID
BILAL AL FAYOUMI
GUILHEM CAUSSE
OMAR HARCHI
AMARA KOUSSAY

29 janvier 2026

Introduction

Quoridor is an abstract strategy board game designed by Mirko Marchesi and first published in 1997 by the French company Gigamic. The game is played on a square grid and is based on two simple actions : moving a pawn and placing walls. Despite this apparent simplicity, the game exhibits a significant strategic depth, as players must constantly balance progress toward their own goal with the obstruction of their opponents.

The objective of this project is to design and implement a complete software version of the game Quoridor, supporting two, three, or four players. The program must strictly respect the official rules of the game and allow players to interact through both a command-line interface and a graphical user interface. In addition, the software must support configuration files, internationalization, and robust error handling, in accordance with the provided specifications.

This project goes beyond the development of a playable game. It aims to apply fundamental software engineering principles, including modular architecture, clear separation of concerns, automated testing, and comprehensive documentation. Quoridor also represents an interesting case study from an algorithmic perspective, as its gameplay can be naturally modeled using graph-based representations and shortest-path algorithms, which are commonly studied in computer science research.



FIGURE 1 – Figure 1 : Gigamic company (left) and Quoridor box (right)

1 Historical Background and Existing Work

Quoridor was invented by Mirko Marchesi in the mid-1990s and officially released in 1997 by the French publisher Gigamic. Since its publication, the game has gained international recognition for its elegant design and educational value, receiving several prestigious awards such as the Mensa Select Award in 1997 and the Games Magazine Game of the Year Award in 1998. Its accessibility, combined with its strategic depth, has contributed to its long-lasting popularity among both casual players and strategy game enthusiasts.

Beyond its commercial success, Quoridor has attracted attention in academic and technical contexts. The game's mechanics, which involve navigating a grid while dynamically placing obstacles, make it a natural candidate for algorithmic modeling. Several studies have explored Quoridor as a testbed for artificial intelligence, pathfinding algorithms, and adversarial search techniques.

In particular, previous academic work has shown that a Quoridor board can be represented as a graph in which vertices correspond to board squares and edges represent legal movements between adjacent squares. Wall placements dynamically modify this graph by removing edges, while pawn jumps introduce temporary alternative connections. Shortest-path algorithms such as breadth-first search or Dijkstra's algorithm are commonly used to evaluate positions, verify the legality of wall placements, and guide decision-making in artificial intelligence agents.

Notable contributions include undergraduate and master-level research that investigate heuristic evaluation functions, graph-based modeling, and search strategies for Quoridor. These works highlight the computational complexity of the game and demonstrate that, despite its simple rules, Quoridor poses non-trivial challenges for both human players and automated systems. This project builds upon these existing ideas while focusing primarily on the design of a robust, extensible, and well-engineered software implementation.

2 Game Classification and Complexity

2.1 Game Classification

Abstract combinatorial strategy game :

- No strong narrative theme and no randomness ; the game relies entirely on logic and player decisions.
- In the case of Quoridor, the game is a pure logical confrontation between players.

Perfect information game :

- There is no hidden information : the complete game state is known to all players at all times.
- In Quoridor, each player can see the positions of all pawns, all placed walls, and the number of walls remaining for each opponent.

Deterministic game :

- Each action leads to a single, well-defined outcome, with no element of chance.
- As a consequence, adversarial search algorithms such as Minimax can be applied, since future game states can be fully predicted from the current state.

2.2 State Space Complexity

According to the thesis of Lisa Glendenning (2005), the state space of Quoridor is estimated to be on the order of 10^{42} legal configurations [?]. For comparison :

- Checkers : approximately 10^{20} states.
- Chess : approximately 10^{46} states.
- Go : approximately 10^{172} states.

The state space complexity of Quoridor is therefore close to that of chess. As a result, it is impossible to store all possible positions in memory or to solve the game by brute-force methods, such as computing the optimal first move exhaustively.

2.3 Game Tree Complexity and Branching Factor

At the beginning of a game, there are $8 \times 8 = 64$ possible positions for placing a wall. Considering the two possible orientations (horizontal and vertical), this yields 128 legal wall placements. In addition, a player typically has up to 3 possible pawn moves. This results in a total of approximately 131 legal moves at the start of the game.

According to Mertens (2006), the average branching factor of Quoridor is around 60, which is relatively high [?]. For comparison : chess has an average branching factor of about 35, and Go has a branching factor of approximately 250. With a branching factor of 60, the size of the search tree grows extremely fast. At depth 3, this leads to roughly $60^3 \approx 216000$ nodes, compared to approximately 42000 nodes in chess.

Technical conclusion : it is essential to use alpha-beta pruning and to design strong heuristic evaluation functions in order to make adversarial search tractable.

3 Rules of the Game

This section describes the official rules of Quoridor as implemented in this project. The objective is to provide a precise and unambiguous description of the game mechanics so that a complete game can be played without interpretation issues.

3.1 Board and Components

Quoridor is played on a square board composed of a 9×9 grid, resulting in 81 playable squares. The squares are connected orthogonally, allowing movement in the four cardinal directions. Between these squares, grooves allow walls to be placed in order to block movement.

Each player controls a single pawn. In addition, a fixed number of walls is available and distributed among the players at the beginning of the game. Walls are barriers that block movement between adjacent squares and play a central role in shaping possible paths across the board.

3.2 Initial Setup

At the beginning of the game, the board is empty except for the players' pawns, which are placed at predefined starting positions.

In a two-player game, one pawn is placed at the center square of the bottom row, while the second pawn is placed at the center square of the top row. Each player's objective is to reach any square on the opposite side of the board.

In a four-player game, pawns are placed at the center of each side of the board : one on the top, one on the bottom, one on the left, and one on the right. Each player must reach the side opposite their starting position.

In the two-player configuration, each player receives ten walls. In the four-player configuration, each player receives five walls in order to maintain balance and avoid excessive blocking.

Players alternate turns, starting with one of the players chosen according to the game configuration.

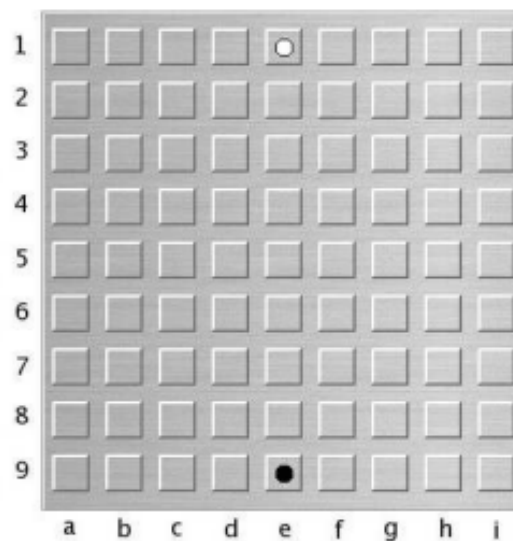


FIGURE 2 – Quoridor board layout

3.3 Pawn Movement

On their turn, a player may move their pawn to an adjacent square. Movement is allowed only in the four orthogonal directions : forward, backward, left, or right. Diagonal movement is not allowed under normal circumstances.

A pawn may only move to an adjacent square if there is no wall blocking the path between the two squares. Pawns cannot move outside the boundaries of the board, nor can they move onto a square already occupied by another pawn.

The goal of pawn movement is to progress toward the target edge of the board while navigating around walls placed by the opponents.

3.4 Jump Rules

If a pawn is located directly adjacent to an opponent's pawn and no wall separates them, the player may jump over the opponent's pawn. The jump moves the pawn to the square immediately beyond the opponent's pawn in the same direction.

If a direct jump is not possible because a wall blocks the square behind the opponent's pawn or because the board edge is reached, the player may instead move diagonally to one of the two adjacent squares to the left or right of the opponent's pawn, provided no wall blocks that diagonal movement.

In four-player games, a pawn may only jump over a single pawn per move. Multiple consecutive jumps over different pawns are not allowed.

These jump rules ensure that pawns cannot be completely blocked by the presence of other pawns and that movement remains possible even in crowded situations.

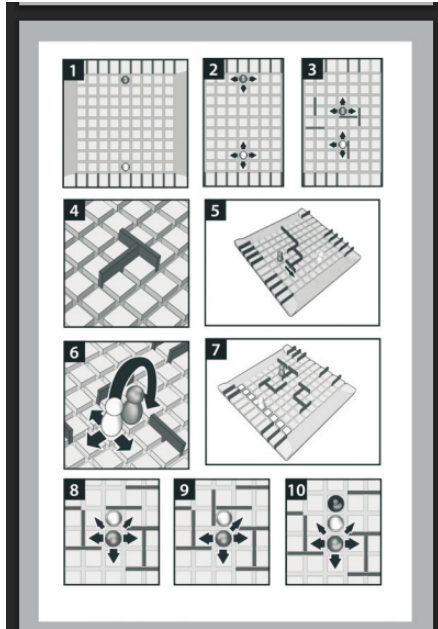


FIGURE 3 – Pawn movement

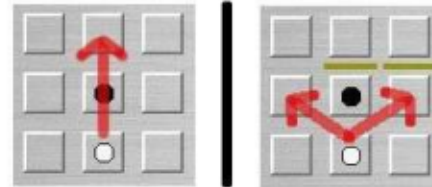


Figure 2.2: Allowed pawn jumps.

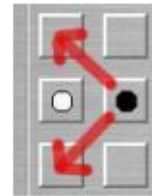


FIGURE 4 – Allowed jump

3.5 Wall Placement

Instead of moving their pawn, a player may choose to place a wall, provided they still have walls available. Walls are placed between squares and must be oriented either horizontally or vertically. Each wall spans exactly two adjacent square connections.

A wall placement must satisfy several constraints. Walls may not overlap or cross existing walls, and they must fit entirely within the board's grooves. Most importantly, a wall may not be placed if it completely blocks all possible paths for any player to reach their target side.

To ensure this rule is respected, at least one valid path from each pawn's current position to its goal edge must remain after the wall is placed. Pawn positions do not block these paths, as pawns can always be jumped over under the movement rules.

Once placed, walls are permanent and cannot be moved or removed for the remainder of the game.

3.6 Victory Condition

The game ends immediately when a player moves their pawn onto any square of their target edge. The first player to reach the opposite side of the board wins the game.

In the standard rules of Quoridor, draws are rare and generally not expected under normal play conditions, as players always have at least one legal move available until a pawn reaches its goal.

4 Algorithms and Data Structures

This section describes the data structures and algorithms selected for the implementation of the Quoridor game. The objective is to justify the chosen representations and algorithmic approaches with respect to the rules of the game and its inherent complexity, rather than to provide implementation-level details.

4.1 Board Representation

The Quoridor board is a fixed-size square grid of dimension 9×9 in the standard configuration. Each square represents a possible position for a pawn, while walls block movement between adjacent squares. Internally, the board

is represented as a structured data object storing the positions of all pawns, the placement and orientation of walls, and the board dimensions.

To support efficient rule validation and path analysis, the board representation must allow fast queries about adjacency relations between squares and the presence of blocking walls. This requirement naturally leads to an abstraction that separates static board geometry from dynamic connectivity.

4.2 Graph-Based Modeling

Following existing academic approaches, the board is modeled as a graph in which each square corresponds to a vertex and each legal movement between adjacent squares corresponds to an edge. Wall placements dynamically modify this graph by removing edges, thereby blocking movement between affected squares.

This modeling approach has been widely adopted in the literature, notably in the work of Glendenning (2005), which formalizes Quoridor as a graph-based game and uses graph traversal algorithms to analyze movement and wall legality [?]. By representing the board as a graph, pathfinding and connectivity constraints can be expressed in a uniform and well-understood framework.

Pawn positions are treated separately from walls in this model. Since pawns can be jumped over according to the rules, they do not permanently block vertices or edges in the graph and are instead handled dynamically during move validation.

4.3 Pawn Movement and Jump Handling

Basic pawn movement corresponds to traversing an edge between two adjacent vertices in the graph. Jump mechanics introduce special movement cases when two pawns are adjacent and no wall separates them. In such situations, a pawn may jump over an opposing pawn to the square immediately beyond it, provided that square is reachable.

If a direct jump is not possible due to a wall or the edge of the board, diagonal movements are permitted under specific conditions. As suggested in Glendenning’s formalization, these jump rules are not encoded directly in the static graph structure but are instead evaluated dynamically when determining the set of legal moves available to a pawn at a given state. This approach keeps the underlying graph representation simple while preserving full rule correctness.

4.4 Wall Placement Validation

Wall placement is subject to strict legality constraints. In addition to geometric restrictions such as non-overlapping and proper alignment, the rules require that every player must retain at least one valid path from their pawn’s current position to their target edge after a wall is placed.

To enforce this constraint, the proposed wall placement is simulated by temporarily removing the corresponding edges from the graph. A reachability test is then performed for each pawn to verify that a path to the goal still exists. This problem naturally reduces to a graph traversal task.

Both breadth-first search (BFS) and Dijkstra’s algorithm are suitable for this purpose. Since all movements have uniform cost, BFS is sufficient and more efficient in practice. This approach is consistent with the methods described in Glendenning (2005) and later works, which rely on shortest-path or reachability checks to validate wall placements.

If at least one path exists for every player, the wall placement is accepted ; otherwise, it is rejected as illegal.

4.5 Shortest Path Evaluation

Shortest-path computations are also useful beyond wall validation. By computing the minimum number of steps required for a pawn to reach its target edge, the system can estimate player progress and relative advantage.

Such evaluations are commonly used in artificial intelligence research on Quoridor, as described by Mertens (2012), where shortest-path distances serve as heuristic features for comparing game states [?]. In the context of this project, these computations may later be reused to support automated players or provide game analysis features.

4.6 Adversarial Search : Minimax and Alpha-Beta Pruning

Quoridor is a perfect-information, deterministic game, making it well-suited for adversarial search algorithms. The Minimax algorithm provides a theoretically optimal strategy for such games by exploring the game tree and selecting moves that maximize the minimum gain while assuming the opponent plays optimally.

However, given Quoridor’s high branching factor (approximately 60 on average) and large state space, an exhaustive Minimax search is computationally intractable beyond very shallow depths. Alpha-beta pruning addresses this limitation by eliminating branches that cannot influence the final decision, thereby reducing the number of nodes evaluated without affecting the optimality of the result.

The alpha-beta algorithm maintains two values during the search :

- **Alpha** : the best value that the maximizing player can achieve so far along the path to the root.
- **Beta** : the best value that the minimizing player can achieve so far along the path to the root.

When beta becomes less than or equal to alpha, the remaining branches at that node can be pruned, as they cannot lead to a better outcome than what has already been found. This pruning can reduce the number of nodes evaluated from $O(b^d)$ to $O(b^{d/2})$ in the best case, where b is the branching factor and d is the search depth.

4.6.1 Heuristic Evaluation Function

Since exploring the entire game tree to terminal states is infeasible, a heuristic evaluation function is used to estimate the value of non-terminal game states. This function assigns a numerical score to a position, typically from the perspective of the current player, where higher scores indicate more favorable positions.

For Quoridor, an effective heuristic should consider multiple factors that influence the game outcome :

- **Distance to goal** : The shortest path distance from each player’s pawn to their target edge, computed using BFS. A player closer to their goal has a positional advantage.
- **Wall resources** : The number of walls remaining for each player. Walls are valuable strategic resources that can be used to block opponents or create advantageous positions.
- **Mobility** : The number of legal moves available to each player. Higher mobility provides more tactical options and flexibility.

The following pseudo-code illustrates a weighted heuristic function that combines these factors :

```
function heuristic(state, player):
    score = 0
    w_dist = 10
    w_walls = 3
    w_mobility = 1

    // 1. Distance to goal
    my_distance = BFS(player)
    opponent_distance = BFS(opponent)
    score = score + w_dist * (opponent_distance - my_distance)

    // 2. Wall resources
    my_walls = remaining_walls_count(player)
    opponent_walls = remaining_walls_count(opponent)
    score = score + w_walls * (my_walls - opponent_walls)

    // 3. Mobility
    my_moves = possible_moves_count(player)
    opponent_moves = possible_moves_count(opponent)
    score = score + w_mobility * (my_moves - opponent_moves)

    return score
```

The weights ($w_{\text{dist}} = 10$, $w_{\text{walls}} = 3$, $w_{\text{mobility}} = 1$) reflect the relative importance of each factor. Distance to goal is weighted most heavily, as it directly relates to the primary objective of reaching the target edge. Wall resources are moderately weighted, as they provide strategic options but are not immediately decisive. Mobility has the lowest weight, as it is a secondary tactical consideration.

This heuristic function is evaluated at the leaves of the search tree (or at a fixed depth limit) and the values are propagated upward using the Minimax algorithm with alpha-beta pruning. The combination of efficient pruning and a well-designed heuristic enables the implementation of competitive automated players while maintaining reasonable computational requirements.

4.7 Monte Carlo Tree Search

As an alternative to the heuristic-heavy Minimax, we implemented the Monte Carlo Tree Search (MCTS) algorithm. MCTS builds an asymmetric game tree based on the results of random simulations (playouts), allowing the AI to focus on promising lines of play without requiring complex domain knowledge. The MCTS cycle consists of four steps : **Selection** : We traverse the existing tree from the root to a leaf node. To balance Exploitation (choosing moves with high win rates) and Exploration (trying less-visited moves), we utilize the UCT (Upper Confidence bounds applied to Trees) formula :

$$UCT = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

Expansion : One or more child nodes are added to the selected leaf node, representing possible future moves.

Simulation (Rollout) : From the new node, the game is played out randomly until a terminal state (win/loss) is

reached.

Backpropagation : The result of the simulation is propagated back up the tree to the root, updating the win counts (w_i) and visit counts (n_i) of all traversed nodes.

This method allows the AI to converge toward optimal play as the number of simulations increases within the allocated time.

4.8 Algorithmic Complexity

Although the overall state space of Quoridor is extremely large, the algorithms used for rule enforcement operate on a fixed-size graph. The number of vertices is bounded by the number of board squares, and the number of edges is limited by local adjacency relations.

As a result, graph traversal algorithms such as BFS run in linear time relative to the size of the board and remain efficient even when executed repeatedly during gameplay. This makes the proposed approach suitable for interactive use and ensures that legality checks do not introduce noticeable delays.

4.9 Summary

The combination of grid-based data structures and graph algorithms provides a robust and conceptually clear foundation for implementing Quoridor. Modeling the board as a graph allows pawn movement, wall placement, and path validation to be handled in a unified manner using well-established algorithmic techniques. This approach is consistent with existing academic work and aligns with the goals of correctness, efficiency, and extensibility.

5 Software Architecture

5.1 Architectural Objectives

The objective of the project architecture is to design a robust, modular, and extensible game engine, while strictly respecting the rules of the game Quoridor. It must ensure good testability of the core logic, facilitate the addition of new user interfaces, and prepare the integration of advanced features such as artificial intelligence or network play.

To achieve these objectives, we adopted a layered software architecture, enabling a clear separation of responsibilities between the different parts of the system.

5.2 Domain Layer

The Domain layer contains all the game rules and data structures. It is fully independent from user interfaces and input/output mechanisms.

It notably includes :

- **GameState**, representing the complete state of a game (board, players, walls, current turn);
- **Board** and **Graph**, responsible for modeling the board as a graph;
- game entities (**Player**, **Wall**, **Position**);
- validation modules (**MoveValidator**, **WallValidator**);
- a pathfinding module (**PathfindingBFS**) used to verify board accessibility after wall placement.

This organization ensures that all game rules are centralized in an independent layer, which simplifies rule validation and enables effective unit testing.

5.3 Application Layer

The Application layer is responsible for orchestrating the overall execution of the game and acts as a coordination point between the user interfaces and the domain logic. This role is mainly fulfilled by the **GameService** class.

The responsibilities of **GameService** include :

- receiving actions produced by the user interfaces;
- triggering their validation through the Domain layer;
- applying valid actions to the game state;
- managing turn progression;
- detecting end-of-game conditions and determining the winner.

This design ensures that any modification of the game state passes through a single control point, guaranteeing the global consistency of the game engine.

5.4 User Interfaces

The user interfaces constitute the outermost layer of the system. They include :

- a command-line interface (CLI);
- a graphical user interface (GUI).

Their role is strictly limited to collecting user inputs, translating them into application-level actions, and displaying the current game state. They contain no domain logic and never directly access the Domain layer.

This separation allows the user interfaces to remain interchangeable without impacting the core game engine.

5.5 Summary

In conclusion, the layered architecture adopted in this project provides a solid foundation for the incremental development of the game engine. It guarantees a clear separation of responsibilities, good testability of the core logic, and extensibility compatible with the future enhancements defined in the project specifications.

6 UML Diagram

The UML diagram below illustrates the detailed architecture of the Quoridor system, presenting the main classes, their attributes, their methods, and the relationships between them.

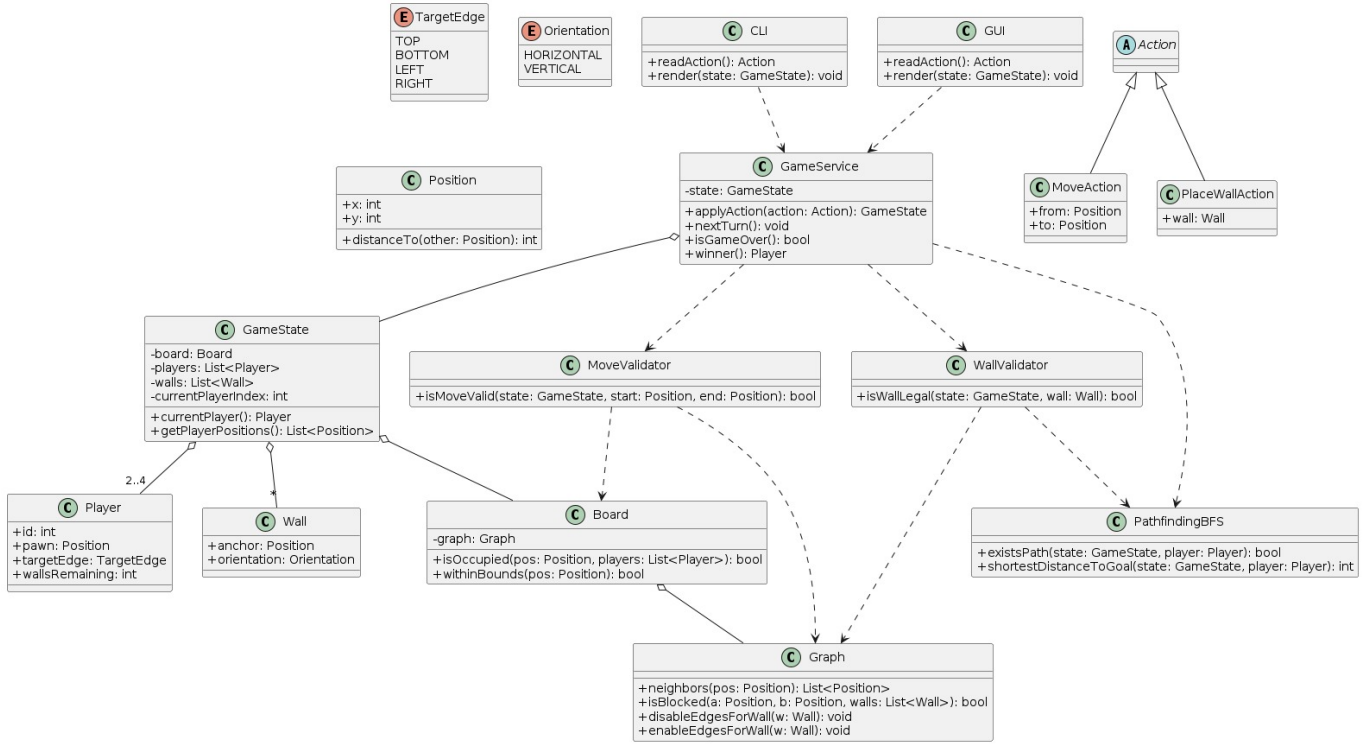


FIGURE 5 – UML class diagram of the Quoridor system

6.1 Diagram Description

The class diagram presents a clearly separated layered architecture, with a distinct separation between the business logic (core), the application layer, and the user interfaces.

6.1.1 Enumerations and Base Types

The diagram defines two essential enumerations :

- **TargetEdge** : defines the four possible target edges (TOP, BOTTOM, LEFT, RIGHT) toward which each player must move.
- **Orientation** : specifies the orientation of walls (HORIZONTAL, VERTICAL).

6.1.2 Main Domain Classes

Position : Represents a coordinate on the board with methods for calculating distances between positions.

Player : Models a player with their identifier, their pawn's position, their target edge, and the number of remaining walls.

Wall : Represents a wall placed on the board, defined by an anchor position and an orientation.

Board : Encapsulates the game board and maintains a reference to an underlying graph for connectivity modeling.

6.1.3 Graph-Based Modeling

Graph : Central class that models the board as a graph, where vertices correspond to squares and edges to legal movements. This class dynamically manages the addition and removal of edges when placing walls, and provides methods to check if a path is blocked.

PathfindingBFS : Implements the breadth-first search algorithm to verify the existence of paths and calculate minimum distances to goals. This class is crucial for validating the legality of wall placements.

6.1.4 Game State and Actions

GameState : Contains the complete game state, including the board, the list of players (2 to 4), the list of placed walls, and the current player's index. This class centralizes all information necessary to represent a game state.

Action : Abstract class representing a generic action. Two concrete classes inherit from it :

- **MoveAction** : represents a pawn movement with a starting position and an ending position.
- **PlaceWallAction** : represents the placement of a wall.

6.1.5 Validation and Business Logic

MoveValidator : Validates pawn movements by checking topological legality, the presence of blocking walls, and jump rules.

WallValidator : Validates wall placements by ensuring they respect geometric constraints and, most importantly, that they do not completely block a player's path to their goal. This validation uses **PathfindingBFS** to verify connectivity.

6.1.6 Application Layer

GameService : Central service that orchestrates the game flow. It maintains the game state, applies actions, manages game turns, and determines end-of-game conditions. This class constitutes the unique entry point for user interfaces.

6.1.7 User Interfaces

CLI and **GUI** : Two distinct user interfaces that share the same responsibility : reading player actions and displaying the game state. They both depend on **GameService** and **GameState**, but do not interfere with the business logic.

6.1.8 Relations and Dependencies

The diagram clearly illustrates the separation of responsibilities :

- Composition relationships (filled diamond) show that **GameService** owns a **GameState**, which itself owns a **Board** and lists of **Player** and **Wall**.
- Dependencies (dashed arrows) indicate that interfaces depend on the application layer, which itself depends on validators and pathfinding, respecting the principle of unidirectional dependency.
- Inheritance (hollow triangle) shows that **MoveAction** and **PlaceWallAction** are specializations of **Action**.

This architecture ensures that business logic remains independent of interfaces, facilitating testing, maintenance, and system extensibility.

Extended Specifications

F20. Pawn Movement and Jump Mechanism

Target functionality : Allow the current player to move their pawn to a free adjacent square, or to perform a jump over an opponent if the conditions are met.

1. Prerequisites and Sub-requirements

Prerequisites :

- **F15 (Interactive Shell)** : The system must be able to receive and parse a command such as "e2-e3" or "e2-e4".
- **F26 (Graph Module)** : The board must be modeled as a graph to verify connectivity between squares.
- **F1 (Game Management)** : The game is started and it is the concerned player's turn.

Sub-requirements :

- **Parsing** : Convert alphanumeric coordinates (e.g. "e2") into graph indices or (x, y) coordinates.
- **Topological Validation** :
 - Simple case : Is the target square adjacent (distance = 1) ?
 - Jump case : Is the target square at distance = 2 (aligned or diagonal) ?
- **Jump Detection** : If the target is at distance 2, verify that an opponent pawn strictly occupies the intermediate square.
- **Wall Verification** : Ensure that no wall blocks the path (neither between the start and the opponent, nor between the opponent and the destination).

2. Functions and Signatures

Name : `is_move_valid`

Role : Validates whether a move (simple movement or jump) is legal according to the current board state and the opponents' positions.

Signature (Python) :

```
def is_move_valid(board: Graph, players: list, start: str, end: str) -> bool:
```

3. Unit Tests (Examples)

- **Test 1 (Simple Move) :**
 - *Input* : Player 1 at "e2", target "e3". No wall, free square.
 - *Output* : **True**.
- **Test 2 (Wall Obstacle) :**
 - *Input* : Player 1 at "e2", target "e3". Horizontal wall present at "e2h".
 - *Output* : **False**.
- **Test 3 (Valid Direct Jump) :**
 - *Input* : Player 1 at "e2", Player 2 at "e3". Player 1 targets "e4" (face-to-face configuration).
 - *Output* : **True**.
- **Test 4 (Blocked Jump) :**
 - *Input* : Player 1 at "e2", Player 2 at "e3". Player 1 targets "e4". A wall is present behind Player 2.
 - *Output* : **False** (The jump is not possible).

F7. Undo and Redo Mechanism

In addition to rule enforcement and path validation, the game engine must support the ability to undo and redo actions. This functionality is essential for several reasons, including user interaction (undoing a mistaken move), artificial intelligence search (backtracking during Minimax exploration), and potential replay or analysis features.

To support undo and redo operations efficiently, the game state evolution is modeled as a sequence of reversible actions rather than irreversible state mutations. Each player action, whether a pawn movement or a wall placement, is represented as an atomic operation that records all the information required to revert its effects.

A stack-based approach is used to manage this mechanism. Two stacks are maintained :

- an undo stack, which stores the history of applied actions ;
- a redo stack, which stores actions that have been undone and may be reapplied.

When an action is executed, it is pushed onto the undo stack and the redo stack is cleared. Undoing an action consists of popping the most recent operation from the undo stack and applying its inverse, thereby restoring the previous game state. Redoing an action consists of popping an operation from the redo stack and reapplying it to the game state.

This design aligns naturally with the requirements of adversarial search algorithms. During Minimax exploration, actions can be applied and reverted efficiently without duplicating entire board states, which significantly reduces memory usage and improves performance. Moreover, this approach maintains a clear separation between game logic and state history management.

By modeling game evolution as a reversible sequence of actions, the undo/redo mechanism contributes to the robustness, testability, and extensibility of the overall system architecture.

Références